

高级语言程序设计

实验报告

南开大学 自动化

姓名 佟佳慕

学号 2410289

班级 自动化班

2026 年 5 月 12 日

目录

1	作业题目	2
2	开发软件	2
3	课题要求	2
4	主要流程	2
4.1	整体流程	2
4.2	算法或公式	3
4.2.1	(1) 双重字母处理—两趟扫描算法 (Board::getRowResult())	3
4.2.2	(2) 翻转动画—Tile::triggerFlip()	3
4.2.3	(3) 弹跳动画—Tile::triggerPop()	3
4.2.4	(4) 抖动动画—Tile::triggerShake()	4
4.2.5	(5) 颜色更新	4
5	单元测试	4
6	收获	5
6.1	1. SFML 3 事件处理机制	5
6.2	2. 两趟扫描处理重复字母	5
6.3	3. 游戏状态机设计	6
6.4	4. 虚函数与多态的实际应用场景反思	6
7	AI 使用报告	6

1 作业题目

Wordle 猜词游戏是基于 C++17 与 SFML 3 开发的图形化单词益智游戏。它遵循经典 Wordle 规则，通过颜色提示反馈猜词结果，在休闲娱乐的同时帮助用户巩固拼写、丰富英语词汇量。程序采用模块化设计，界面流畅、功能完整，兼具实用性与学习价值。

2 开发软件

- **IDE:** Visual Studio Code
- **构建工具:** CMake 3.16+
- **编译器:** Apple Clang (C++17)
- **图形库:** SFML 3 (Graphics, Window, System 模块)
- **平台:** macOS (跨平台支持 Linux / Windows)

3 课题要求

1. 面向对象设计，清晰的类层次结构。
2. 图形化界面交互，包含键盘输入与鼠标点击。
3. 动画效果（弹跳、翻转、抖动）。
4. 词库加载与单词有效性验证。
5. 重复字母正确处理。

4 主要流程

4.1 整体流程

实现思路：

定义 4 个核心类：

- **Game:** 游戏主循环、事件处理、渲染调度、UI 组件（按钮、确认对话框、键盘绘制、覆盖层）。
- **Board:** 6×5 网格管理、当前行/列追踪、单词提交与结果计算。
- **Tile:** 单个格子的状态、动画（弹跳/翻转/抖动）与绘制。
- **Dictionary:** 词库加载（`unordered_set` 实现 $O(1)$ 查词）、随机目标词选取。

游戏循环采用标准 SFML 模式：`processEvents()` → `update(dt)` → `render()`。

主循环中根据 `ConfirmAction` 枚举状态机控制确认对话框的显示与隐藏；`waitForAnimation` 和 `gameOver` 状态控制输入屏蔽。

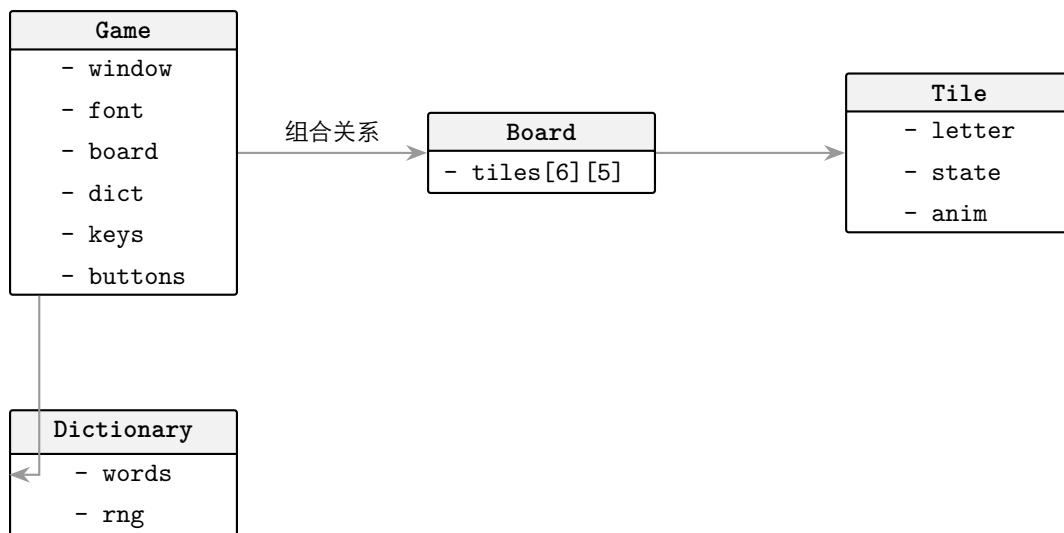


图 1: Wordle 游戏核心类关系图

4.2 算法或公式

4.2.1 (1) 双重字母处理—两趟扫描算法 (Board::getRowResult())

- **第一趟**：标记所有位置完全匹配的字母为 **Correct**，并锁定对应目标位置。
- **第二趟**：对于非 **Correct** 的猜测字母，在未锁定的目标位置中查找是否存在该字母，若存在则标记为 **Present**，并锁定该目标位置。
- 剩余字母标记为 **Wrong**。
- **意义**：此法正确处理了目标词中存在重复字母的场景（如 **ABBEY** 猜 **BABES**）。

4.2.2 (2) 翻转动画—Tile::triggerFlip()

使用余弦函数模拟翻转效果：

$$scaleY = \cos(t \times \pi)$$

其中 $t = \text{flipTime} / \text{FlipDuration}$ ($0.0 \sim 1.0$)。当 $scaleY < 0$ 时切换为结果颜色并取绝对值，模拟从背面翻到正面的效果。动画时长 0.5s，每个格子依次延迟触发。

4.2.3 (3) 弹跳动画—Tile::triggerPop()

输入字母时触发，缩放曲线分两段：

- 前半段 ($t < 0.5$)： $0.8 \rightarrow 1.1$
- 后半段 ($t \geq 0.5$)： $1.1 \rightarrow 1.0$

动画时长 0.15s，模拟按键弹跳效果。

4.2.4 (4) 抖动动画—`Tile::triggerShake()`

无效单词提交时触发，水平偏移公式：

$$\text{offset} = \sin(t \times 30) \times 8 \times \left(1 - \frac{t}{\text{duration}}\right)$$

使用衰减正弦波（0.4s），强度随时间线性减小，同时格子变红（#e53935）提示错误。

4.2.5 (5) 颜色更新

提交猜测后更新虚拟键盘颜色，优先级：Correct > Present > Wrong > Empty。不降低已标记的 Correct 级别（Correct 一旦标绿永不退回）。

5 单元测试

针对每个核心功能设计手工测试案例，覆盖正常输入、无效单词、游戏胜利、游戏失败、放弃操作、重新开始、重复字母处理等场景。针对确认对话框，测试鼠标点击按钮和 **Escape** 键取消两种交互方法。

表 1: 单元测试用例表

序号	操作	预期结果	结果
1	点击 A-Z 字母键	字母依次填入当前行，伴随弹跳动画	通过
2	按 Backspace 键	删除当前行最后一个字母	通过
3	输入 4 个字母按 Enter	无反应（行未满）	通过
4	输入无效 5 字母单词按 Enter	行抖动闪烁红色，提示”Not in word list”	通过
5	输入有效猜测单词按 Enter	行依次翻转显示结果颜色	通过
6	猜中目标词	显示”You Win!” 覆盖层及目标词	通过
7	6 次未猜中	显示”Game Over” 覆盖层及目标词	通过
8	点击 Restart 按钮	弹出”Restart game?” 确认对话框	通过
9	确认对话框中点击 Yes	新游戏开始，棋盘、键盘颜色全部重置	通过
10	确认对话框中点击 Cancel	返回游戏，棋盘状态不变	通过
11	点击 Give Up 按钮	弹出”Give up?” 确认对话框	通过
12	确认放弃	显示”You gave up!” 覆盖层及目标词	通过
13	游戏结束后点击覆盖层 Restart 按钮	弹出确认对话框，确认后新游戏开始	通过
14	确认对话框显示时按字母键	被屏蔽，不输入到棋盘	通过
15	确认对话框显示时按 Escape	对话框关闭，返回游戏	通过
16	动画播放期间按键输入	被 <code>waitingForAnimation</code> 屏蔽	通过
17	重复字母目标词（如 ABBEY）	Correct/Present/Wrong 标记正确，不重复计数	通过

6 收获

6.1 1. SFML 3 事件处理机制

代码片段：

```
1 while (const std::optional<sf::Event> event = window.pollEvent()) {
2     if (event->is<sf::Event::Closed>()) {
3         window.close();
4         continue;
5     }
6     if (const auto* mb = event->getIf<sf::Event::MouseButtonPressed>()) {
7         handleMouseClick(mb->position.x, mb->position.y);
8         continue;
9     }
10    if (const auto* te = event->getIf<sf::Event::TextEntered>()) {
11        // 处理文字输入
12    }
13 }
```

问题分析：

SFML 2 使用 `while (window.pollEvent(event))` 并配合 `event.type == sf::Event::TextEntered` 进行判断。SFML 3 改用 `std::optional` 配合 `getIf<T>()` 模式匹配，类型更安全，避免了传统 `union` 方式可能带来的内存访问错误。

6.2 2. 两趟扫描处理重复字母

代码片段：

```
1 // First pass: mark correct positions
2 for (int i = 0; i < Cols; ++i) {
3     if (guess[i] == target[i]) {
4         result[i] = Tile::State::Correct;
5         targetUsed[i] = true;
6     }
7 }
8 // Second pass: mark present for letters that exist elsewhere
9 for (int i = 0; i < Cols; ++i) {
10    if (result[i] == Tile::State::Correct) continue;
11    for (int j = 0; j < Cols; ++j) {
12        if (!targetUsed[j] && guess[i] == target[j]) {
13            result[i] = Tile::State::Present;
14            targetUsed[j] = true;
15            break;
16        }
17    }
18 }
```

问题分析：

如果只用单趟扫描,当目标词 **ABBEY** 遇到猜测词 **BABES** 时,第一个 **B** 可能被错误标记为 **Correct**,或者两个 **B** 都被错误标记为 **Present**。两趟扫描先锁定位置完全匹配的字母,再在剩余未匹配位置中匹配字母,确保每个目标字母只被使用一次,完美解决了重复字母的计数问题。

6.3 3. 游戏状态机设计

问题分析：

在初期实现中,按钮点击直接执行操作,没有二次确认步骤,容易造成误触。引入 **ConfirmAction** 枚举作为状态机管理后,交互流程变为: 点击按钮 → 设置 **pendingConfirm** 状态 → 弹出对话框 → 用户确认或取消 → 清除状态。当 **pendingConfirm != None** 时,程序会屏蔽所有底层的游戏输入,仅响应对话框相关的交互操作 (**Yes / Cancel / Escape**),形成了清晰的输入分层保护机制。

6.4 4. 虚函数与多态的实际应用场景反思

虽然此项目未直接使用虚函数继承体系(遵循了“组合优于继承”的原则),但从分析项目整体结构可知:**Game** 类持有 **Board**、**Dictionary**、**sf::RenderWindow** 等对象作为其成员(属于显式的组合关系),而非通过继承延伸。这种设计使得各模块职责清晰、耦合度极低。如果未来项目需要支持不同的游戏模式(例如无限模式、限时模式),便可以通过抽象出一个 **GameMode** 基类,并配合虚函数来实现经典的多态扩展。

7 AI 使用报告

本报告说明 Wordle 单词猜谜程序的 AI 使用情况,该程序核心功能手动编码,仅部分模块使用 Deepseek 辅助,属于部分使用 AI 模式。程序核心用途为用户通过 6 次猜测,根据颜色提示猜出 5 字母英文单词。Deepseek 仅用于两个辅助环节:一是开发阶段辅助生成、筛选 5 字母合法单词库,简化手动整理工作量,最终单词库经手动校验确定;二是辅助优化答案随机选取算法,确保随机性。程序核心功能均无 AI 参与,包括用户输入校验、猜测结果反馈、游戏循环、交互展示等核心逻辑,以及程序启动、异常处理等底层运行环节。Deepseek 仅为开发辅助工具,不参与程序运行,不涉及复杂 AI 能力及数据上传,核心功能稳定性依赖手动编码,符合部分使用 AI 的开发需求。